

E-commerce Backend — Interview Q&A

Generated: June 1, 2026

1. **Q1: Project purpose**
A1: Provide a TypeScript/Node backend for an e-commerce app: product catalog, user auth, cart/wishlist, order lifecycle, admin reports, and email notifications.
2. **Q2: Entry points**
A2: *src/index.ts* boots the server and connects DB; *src/app.ts* builds the Express app, registers middleware and routes, and attaches the error handler.
3. **Q3: Authentication flow**
A3: Middleware *current-user-middleware.ts* parses the token/session and attaches *req.currentUser*. *require-auth-middleware.ts* enforces protected routes. Password hashing via *password.service.ts*.
4. **Q4: Database layer**
A4: Mongoose + MongoDB with models in *src/models/* (User, Product, Cart, Order). *mongoose.service.ts* centralizes connection logic.
5. **Q5: Error handling**
A5: Custom error classes in *src/common/src/errors/* (e.g., *CustomError*, *RequestValidationError*) and a single *error-handler-middleware.ts* formats responses.
6. **Q6: Adding a new route**
A6: Create a router under *src/routes/* or *src/admin/*, add validators, implement handler using models/services, register router in *src/app.ts*, apply auth/validation middleware.
7. **Q7: Password security**
A7: *password.service.ts* wraps a hashing algorithm (bcrypt or similar) for hashing on sign-up and comparing on login.
8. **Q8: Background jobs**
A8: *expiration-queue.service.ts* implements expiration logic (timeouts/intervals or a queue); *email.service.ts* handles notification delivery.
9. **Q9: Admin vs public routes**
A9: Admin routes are under *src/admin/*, protected by auth middleware and likely role checks in handlers.
10. **Q10: Input validation**
A10: *validate-request-middleware.ts* uses validators (express-validator or similar) and throws *RequestValidationError* on failures.
11. **Q11: Scaling limitations**
A11: Single-node memory-bound expiration/email implementations; no distributed job queue; unclear rate-limiting; no caching layer shown.
12. **Q12: Security concerns to review**

A12: Audit secrets/env management, HTTPS, CORS, token expiry, brute-force protections, and safe error messages.

13. **Q13: Testing strategy**

A13: Unit tests for services/models (Jest), integration tests for routes (supertest), E2E flows via Postman/Cypress with a test DB instance.

14. **Q14: Deployment**

A14: Docker via *server.dockerfile* and *docker-compose.yml*. CI builds images, pushes to registry, and deploys to orchestrator.

15. **Q15: Performance improvements**

A15: Add Redis caching, DB indexes, connection pooling, worker queue for email/expiration, and pagination.

16. **Q16: Data modeling**

A16: Relations via references/embedded docs; orders store product snapshots to preserve history.

17. **Q17: Migration strategy**

A17: Add fields with defaults, write migration scripts to backfill, deploy backwards-compatible code, then remove old fields later.

18. **Q18: Observability**

A18: Add structured logging, tracing (OpenTelemetry), metrics (Prometheus), and log aggregation (ELK/Datadog).

19. **Q19: Explain a router**

A19: *src/routes/order/create-order.router.ts* validates checkout, converts cart to order, computes totals, persists *Order*, adjusts stock, and triggers email/cleanup.

20. **Q20: Design tradeoffs**

A20: Admin in same repo simplifies deployment and reuse but increases coupling and attack surface; consider microservice split for scale.

Notes: Use the project's Postman collection to test routes. Generated by Copilot assistant.